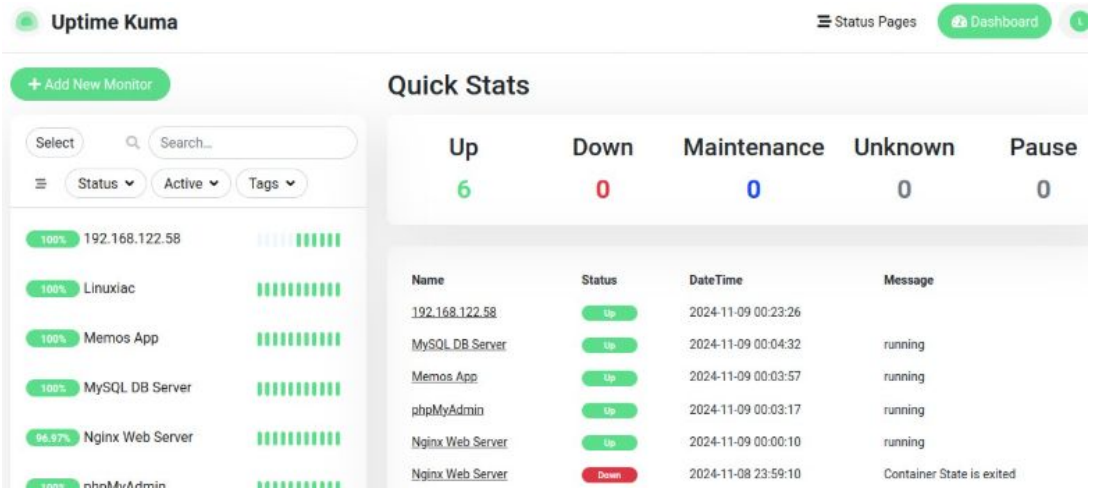
docker-compose Monitoring

A.SCHUTZ

Uptime Kuma

[Webhook Discord Uptime Kuma]

Uptime Kuma sera votre allié pour monitorer la santé de vos API et créer des alertes rapide




```yaml
uptime-kuma:
  image: louislam/uptime-kuma:1
  container_name: uptime_kuma
  ports:
    - "3001:3001"
  volumes:
    - uptime-kuma-data:/app/data
  networks:
    - monitor_network
  restart: unless-stopped
```

grafana - prometheus

- prometheus:
  - comment enregistrer des métriques temporelles d'une API ou d'une Application en python
  - `prometheus/prometheus.yml`
- grafana :
  - provisioning/dashboards
    - `dashboards.yml`
    - `dashboards.json`
  - provisioning/datasources
    - `datasources.yml`

Exercice

Cloner:

`https://github.com/DeVerMyst/docker-compose-prometheus-grafana.git`

Conclusion sur docker-compose

Avantages :

- **Simplicité d'utilisation :** Idéal pour les environnements de développement et de test locaux.
- **Facile à configurer :** Utilisation de fichiers YAML simples.

Parfait pour les applications à conteneurs multiples sur un seul hôte.

Conclusion

Inconvénients :

- N'est pas conçu pour l'orchestration à grande échelle.
- Manque de fonctionnalités avancées (auto-guérison, scalabilité automatique).
- Pas adapté aux environnements de production complexes.

Conclusion

Quand l'utiliser :

- **Développement local :** Pour mettre en place rapidement des environnements de développement.
- **Tests :** Pour créer des environnements de test isolés.

Applications simples sur un seul serveur.

Mais surtout

B1

- Base de données
- Api et modules
- santé du conteneur
- monitoring de l'API et des ressources
- visualisation des logs

B2

- Service d'IA (api)
- LLM éventuellement
- santé du conteneur
- monitoring du modèle, de l'API et des ressources
- visualisation des logs

B3

- Base de données
- Application
- santé du conteneur
- monitoring de l'application et des ressources
- visualisation des logs